

SILK++

THE BUILD BIBLE

Clone Rumik.ai Silk · Surpass Hume EVI 3 · Out-latency ElevenLabs Flash

Targets on the kill-list

■ ElevenLabs v3	75 ms TTFB · UTMOS 4.05 · weak code-switch
■ Hume EVI 3	1200 ms e2e · implicit emotion · no full-duplex audio out
■ Rumik Silk 1	~1000 ms · SFT-only · best Hinglish today
■ Sarvam Bulbul	100 ms · basic emotion · 4 Indic

SILK++ targets

≤ 80 ms	first-packet TTFB
≤ 350 ms	voice→voice end-to-end
≥ 4.25	UTMOS naturalness
3 sec	zero-shot voice clone
\$0.04	cost / 1M chars
Apache 2.0	open weights, full duplex

01 · Executive Briefing

Mission. Ship — within 16 weeks — an open-source emotional voice AI that (a) clones Rumik Silk's Hinglish + code-switch superpower, (b) beats Hume EVI 3 on controllable emotion, (c) matches ElevenLabs Flash first-packet latency on Indian infra, and (d) does it with open weights under Apache 2.0.

Thesis. Silk's moat is not the architecture — it is the Hinglish + inline-tag dataset + EmoVoice-style emotion control layered on a forked Sesame CSM. We replicate that, then drive three knives in.

The Three Knives

#	Innovation	What it kills	How
A	PCS-Tok — phoneme-anchored unified tokenizer	Silk's code-switch lead	epitran + IndicXlit anchors every Latin-script Hinglish word to its IPA, so 'main kal milunga' and '■■■■ ■■ ■■■■■■■■' share embeddings.
B	HEC — Hierarchical Emotion Control (FiLM + tags + V/A/D)	Hume's empathic edge	FiLM γ, β modulation per transformer layer driven by discrete tags + continuous valence/arousal/dominance vectors + duration.
C	DPO + GRPO two-stage prosody alignment	Silk's SFT-only ceiling	100 k human-rated preference pairs (Surge AI 10 k + GPT-4o-audio judge 90 k) → DPO $\beta=0.1$ → GRPO with composite reward.

Capability Battle Card

Metric	ElevenLabs v3	Hume EVI 3	Rumik Silk 1	Sarvam Bulbul	SILK++ target
First-packet latency	~75 ms	~300 ms	~1000 ms	~100 ms	≤ 80 ms
End-to-end voice→voice	~500 ms	~1200 ms	<1000 ms	~600 ms	≤ 350 ms
UTMOS naturalness	4.05	4.10	~3.95	~3.85	≥ 4.25
Code-switch (Hinglish)	Weak	Poor	Best today	Good	SOTA, 4 mixes
Emotion control	Tags	Implicit+RL	Inline tags	Basic	Hierarchical+R LHF
Zero-shot voice clone	5 s	Persona only	Unknown	Limited	3 s
Cost / 1M chars	\$0.18	\$0.20+	Unknown	\$0.10	\$0.04
Open weights	No	No	No	Partial	Yes (Apache 2.0)
Full duplex	No	Yes	No	No	Yes

Read this twice. Rumik almost certainly forked Sesame CSM-1B and added data, not novel architecture. Forking saves you 4 months and ~\$300 k vs pretraining. Budget 70 % of your effort on data + alignment, not on inventing new layers.

02 · Enemy Dossiers

ElevenLabs v3 / Flash v2.5

What it is. Closed-source diffusion-prior + transformer TTS. Flash v2.5 achieves ~75 ms TTFB but on US/EU PoPs only.

Where it bleeds. Weak Hinglish code-switch · expensive (\$0.11/min) · no full-duplex · proprietary lock-in.

How we cut. Match 75 ms on Indian PoP (Mumbai + Singapore), beat on code-switch, undercut by 4×, open the weights.

Hume EVI 3

What it is. Empathic-voice-interface — measures Valence/Arousal/Dominance on the user side, generates 'emotional' speech reactively. Practical voice→voice ~1.2 s.

Where it bleeds. Latency · no native Indic · no inline output control · prosody black-box.

How we cut. Hierarchical Emotion Control with explicit V/A/D and inline tags on the OUTPUT — controllable, not just reactive. Plus 3.4× lower e2e.

Rumik Silk 1

What it is. Almost certainly a fork of Sesame CSM-1B fine-tuned on a custom Hinglish/Tanglish/Manglish/Kanglish corpus with inline emotion tags. SFT only.

Where it bleeds. SFT-only ceiling · ~1 s TTFB · single-track (TTS, not full-duplex) · closed weights.

How we cut. Add DPO + GRPO (Fish-S2 recipe), drop TTFB 12×, ship full-duplex via Moshi-style multi-stream, release Apache 2.0.

Sarvam Bulbul / AI4Bharat

What it is. Indic-first multilingual TTS, decent latency, partial open weights.

Where it bleeds. Basic emotion · limited code-switch · weak voice clone.

How we cut. Match coverage (10 Indic) + add hierarchical emotion + 3-s clone + sub-80 ms TTFB.

03 · Architecture — Layers L0 → L7

SILK++ is an 8-layer stack. Every layer has a measurable Definition-of-Done. A coding agent reading this PDF can build, train, and serve every layer without further human guidance.

Layer	Role	Key tech		DoD
L7 Client	WebRTC / WebSocket	Pipecat + LiveKit		<200 ms jitter, Opus 20 ms frames
L6 Pipeline	VAD → (ASR) → DialogLLM → TTS	Silero v5 + LLM + SILK++		End-to-end e2e ≤ 350 ms p50
L5 Serving	Custom audio sampler	vLLM fork + Triton kernel		31-codebook fused head, FP8
L4 Fast Decoder	Predicts cb1..cb31 per frame	350 M Llama-mini, 6 layers, dim 1024		Single forward, 12 ms / frame
L3 Slow Backbone	Predicts cb0 @ 12.5 Hz	8 B Llama-3 (or 1 B CSM fork) + FiLM		WER round-trip ≤ 4 % EN, ≤ 8 % Hinglish
L2 Tokenizers	Text + audio discretization	PCS-Tok (64 k BPE) + Mimi-Lite (32 RVQ)		PESQ ≥ 4.0, code-switch F1 ≥ 0.90
L1 Conditioning	Persona + voice ref + emotion	ECAPA + WavLM + V/A/D + tags		3-s clone speaker-sim ≥ 0.78
L0 Data	5 tiers of corpora	T0–T5 (codec / pretrain / CS-SFT / emo / DPO / instr)		≥ 80 k hrs T1, 100 k DPO pairs

TTFB Budget (target ≤ 80 ms, H100 SXM, FP8)

Stage	Budget	How
BPE tokenize	1 ms	sentencepiece, pre-warmed
Backbone first-frame	35 ms	CUDA graphs, FP8, KV-cache pre-warm, speculative draft
Depth decoder (cb1..31)	12 ms	Parallel heads in one Triton launch
Mimi-Lite frame	decode 1 18 ms	Streaming convs, fused kernel, separate CUDA stream
Network + jitter	14 ms	LiveKit Opus 20 ms, Mumbai PoP
TOTAL	≤ 80 ms	Beats ElevenLabs Flash in-India practical

Verified architecture truths (from primary sources, cited in §16): (1) Mimi = 24 kHz → 12.5 fps, 32 RVQ levels, cb0 distilled from WavLM-Large layer 7, trained with adversarial + spectral + commitment loss + RVQ dropout. (2) Moshi Depth Transformer = 6 layers, dim 1024, 16 heads. (3) Fish-S2 Slow AR = Qwen3-4B, Fast AR = 4 layers; trained with GRPO over $R_{total} = \lambda_{STT} \cdot R_{STT} + \lambda_{Pref} \cdot R_{Pref} + \lambda_{SIM} \cdot R_{SIM}$.

04 · Reference Stack — Exact Repos & Versions

Every artifact below has a verified URL, a pinned commit/tag, and a license. Coding agents must use these exact references; introducing new dependencies requires a flagged PR per the agent execution protocol (§13).

Component	Repo / HF	Version	License	Purpose
Sesame CSM	github.com/SesameAILabs/csm	main (≥ 2025-05-20)	Apache 2.0	Backbone reference impl
CSM-1B weights	hf: sesame/csm-1b	latest	Apache 2.0	Starting weights
Llama-3.2-1B	hf: meta-llama/Llama-3.2-1B	latest	Llama 3.2	CSM text backbone
Mimi codec	hf: kyutai/mimi	latest	CC-BY-4.0	Audio codec (frozen; retrained → Mimi-Lite)
Moshi	github.com/kyutai-labs/moshi	v0.2.4+	Apache/MIT	Full-duplex reference
Helium 7B	hf: kyutai/helium-1-preview-7b	latest	CC-BY-4.0	Optional bigger backbone
Fish-Speech S2	github.com/fishaudio/fish-speech	s2 branch	CC-BY-NC-SA	Dual-AR + GRPO reference
OpenAudio S1	fishaudio/fish-speech	openaudio-s1	Apache 2.0	Commercial fallback
VoXstream	github.com/herimor/voxstream	voxstream	MIT	102 ms streaming reference
WavLM-Large	hf: microsoft/wavlm-large	latest	MIT	Semantic distill target
ECAPA-TDNN	hf: speechbrain/spkrec-ecapa-voxceleb	latest	Apache 2.0	Speaker encoder
Whisper-large-v3	hf: openai/whisper-large-v3	latest	MIT	ASR back-translation
UTMOS	github: sarulab-speech/UTMOS22	main	—	MOS scoring
Silero VAD v5	hf: snakers4/silero-vad	v5	MIT	Sub-1 ms VAD
IndicXlit	github: AI4Bharat/IndicXlit	main	MIT	Indic transliteration
epitran	github: dmort27/epitran	main	MIT	Grapheme → IPA
vLLM (fork)	github: vllm-project/vllm	v0.7+	Apache 2.0	Serving + custom audio kernel
Pipecat	github: pipecat-ai/pipecat	main	BSD-2	Streaming pipeline
LiveKit Agents	github: livekit/agents	main	Apache 2.0	WebRTC transport
Unsloth	github: unslothai/unsloth	main	Apache 2.0	2× faster LoRA
TRL (DPO)	github: huggingface/trl	v0.11+	Apache 2.0	DPO trainer
Lhotse	github: lhotse-speech/lhotse	main	Apache 2.0	Audio manifests
WebDataset	github: webdataset/webdataset	main	BSD-3	Sharded streaming
EmoVoice	arXiv 2504.12867	v3	paper	Emotion control recipe
SpeechAlign	arXiv 2404.05600	—	paper	DPO for speech

05 · Repo Skeleton — Every File, Every Folder

This is the canonical filesystem tree. Coding agents create exactly this structure via `scripts/00_bootstrap.sh`. Do not add or rename folders without a PR.

```
silk-plus-plus/
├── README.md
├── pyproject.toml           # uv / PEP 621
├── LICENSE                 # Apache-2.0
├── .python-version         # 3.11.9
├── .pre-commit-config.yaml
├── configs/
│   ├── base.yaml          # Hydra root
│   ├── model/
│   │   ├── slow_8b.yaml
│   │   ├── slow_1b_csm.yaml # cheap-path fork
│   │   ├── fast_350m.yaml
│   │   ├── codec_mimi_lite.yaml
│   │   └── tokenizer_pcs.yaml
│   ├── data/ {tier0..tier4}.yaml
│   ├── train/ {stage1..stage7}.yaml
│   ├── infer/ {streaming, edge_4bit}.yaml
│   └── personas/ swati_mumbai_genz.yaml, arjun_delhi_gym.yaml, ... (10)
├── silkpp/
│   ├── codec/             # Mimi-Lite
│   │   ├── encoder.py · decoder.py · rvq.py · wavlm_distill.py
│   │   └── discriminator.py · losses.py · mimi_lite.py
│   ├── tokenizer/         # PCS-Tok
│   │   ├── pcs_tok.py · phoneme_anchor.py · emotion_tags.py · train_bpe.py
│   ├── model/            # Backbone + Decoder
│   │   ├── slow_backbone.py · fast_decoder.py · emotion_film.py
│   │   ├── speaker_cond.py · persona_embed.py · inner_monologue.py · silkpp.py
│   ├── data/
│   │   ├── manifests.py · webdataset_shards.py · mimi_tokenize.py
│   │   ├── pcs_tokenize.py · synthesize_codeswitch.py · emotion_studio_loader.py
│   │   ├── dpo_pair_gen.py · filters.py
│   ├── training/
│   │   ├── stage1_codec.py ... stage7_grpo.py · compute_amortize.py
│   │   ├── lora_cfg.py · distill_edge.py
│   ├── inference/
│   │   ├── streaming_server.py · vllm_audio_kernel.py · pipecat_pipeline.py
│   │   ├── livekit_room.py · edge_runner_onnx.py · kv_cache_warmer.py
│   ├── eval/
│   │   ├── utmos.py · dnsmos.py · wer_back.py · emotion_classifier.py
│   │   ├── speaker_sim.py · latency.py · code_switch_acc.py · leaderboard.py
├── scripts/
│   ├── 00_bootstrap.sh · 01_download_open_weights.sh · 02_download_open_data.sh
│   ├── 03_train_pcs_tokenizer.sh · 04_train_mimi_lite.sh · 05_tokenize_corpus.sh
│   ├── 06_stage2_backbone.sh ... 14_stage7_grpo.sh
│   ├── 15_distill_edge.sh · 16_serve_vllm.sh · 17_run_eval_suite.sh
├── tests/ (codec_roundtrip, pcs_tokenizer, emotion_parser, film, latency)
├── docker/ (Dockerfile.train, Dockerfile.serve, compose.yaml)
├── infra/ (terraform/, k8s/, modal/)
└── docs/ (ARCHITECTURE.md, DATA.md, TRAINING.md, SERVING.md, EVAL.md)
```

06 · Data Strategy — Tier 0 → Tier 5

Silk's moat is data, not model. We replicate Silk's Hinglish corpus, then exceed it on scale, on language coverage, and on emotion fidelity. Budget 70 % of your effort here.

Tier	Goal	Hours	Sources
T0	Codec pretrain	50 k	LibriHeavy · Emilia-EN · MLS-EN · MUSAN noise · OpenSLR-28 RIRs
T1	Backbone pretrain	80 k	Common Voice 17 · IndicVoices · Shrutilipi · Vaani · Emilia-Multi · YT-India scraped
T2	Code-switch SFT	3 k synth + 200 hrs human-edited	GPT-4o-mini dialogues → Fish-S2 self-host TTS → Whisper-v3 WER filter ≤ 12% → UTMOS gate ≥ 3.6
T3	Emotion studio	1.5 k	50 actors × 30 hrs · 9 emotions · 25 M / 25 F · 9 regional accents · 48 kHz / 24-bit
T4	DPO pairs	100 k pairs	Surge AI 10 k human-rated + GPT-4o-audio judge 90 k synthetic
T5	Instruction-tune	5 k synthetic	(text-instruction, audio-response) — 'speak slower', 'more cheerful', 'Tamil-English mix please'

Tier-2 Code-Switch Synth Pipeline (the Silk killer)

```
# silkpp/data/synthesize_codeswitch.py – execution shape
THEMES = ["chai date", "office wfh", "gym session", "auto ride", "Diwali fight",
          "monsoon nostalgia", "exam stress", "Bumble swipe roast", ...]
EMO_SETS = [{"<excited>","<laugh>"}, {"<sad>","<sigh>"}, {"<flirty>","<whisper>"}, ...]
LANG_PAIRS = [{"hi-Latn","en"}, {"ta-Latn","en"}, {"te-Latn","en"},
              {"bn-Latn","en"}, {"mr-Latn","en"}, {"ml-Latn","en"},
              {"kn-Latn","en"}, {"pa-Latn","en"}, {"gu-Latn","en"}]

# 1. GPT-4o-mini generates 200 k dialogues w/ inline tags (~$300)
# 2. Fish-S2-Pro self-hosted synthesizes audio (~$0, ~14 GPU-days)
# 3. Whisper-large-v3 round-trip ASR → WER < 12%
# 4. UTMOS22 gate → keep top 70%
# 5. WebDataset .tar shards (~50 GB)
```

Tier-3 Studio SOP (the last 30 % of quality)

200 utterances × 9 emotions × 50 actors = 90 k clips · 48 kHz / 24-bit · treated room · Neumann TLM 103 · Apollo Twin X · █6,000 / hr × 30 hrs / actor ≈ █90 L total (~\$108 k). Sign two studios in parallel (Mumbai + Bangalore) for redundancy.

07 · L2 — Tokenizers & Mimi-Lite Codec

7.1 PCS-Tok — Phonetic Code-Switch BPE

64 k BPE trained on a phoneme-anchored corpus. Each word is mapped to IPA via epitran (Devanagari) or IndicXlit (Latin-script Indic), then BPE is trained over the phoneme stream. The model learns 'main kal milunga' and '■■■■ ■■ ■■■■■■■■' as near-identical sequences.

```
special tokens:
  <lang:en>, <lang:hi-Latn>, <lang:hi-Deva>, <lang:ta-Latn>, ...  (22 langs × 2 scripts)
  <speaker:N>, <persona:slug>
  <emotion:laugh>, <emotion:laugh:soft duration=0.4>
  <pause:200ms>, <pause:600ms>
  <PAD>, <EPAD>, <BOS>, <EOS>, <BOA>, <EOA>

training: sentencepiece BPE, vocab=64000, character_coverage=0.9999, 1 hr on 1 CPU
```

7.2 Mimi-Lite Codec (verified specs)

Spec	Value	Source
Sample rate	24 kHz	Kyutai codec explainer
Frame rate	12.5 fps (80 ms / frame)	Kyutai · Moshi §3
Latent dim	512	Moshi paper
RVQ levels	32 (1 semantic + 31 acoustic)	Kyutai
Codebook size	2048 per level	Moshi paper
Semantic distill target	WavLM-Large layer 7	Kyutai
Recon loss	Mel L1+L2 + adversarial (MPD+MSD) + commitment (β=0.25)	Kyutai
RVQ dropout	$k \in [8, 32]$ uniform per batch	Kyutai
Encoder	SeaNet (4 ResBlocks × 4 downsample) + 8-layer Transformer bottleneck	Moshi
Decoder	Mirror SeaNet + HiFi-GAN vocoder head	Moshi
Params	~80 M	Moshi
Training	1 M steps · lr 8e-4 · bs 128 · 12-s windows · 8×H100 · ~14 days	Moshi paper
DoD	PESQ ≥ 4.0 · STOI ≥ 0.95 · ViSQOL ≥ 4.3 · — WavLM-distill cos ≥ 0.85	

Cheap-path note. If timeline is tight, freeze Kyutai's released Mimi and skip Stage 1 entirely. You lose ~5 % quality on Indic but save 14 GPU-days. Rumik almost certainly does this.

08 · L3-L4 — Slow Backbone + Fast Decoder

8.1 Slow Backbone (8 B Llama-3)

Spec	Value
Hidden dim	4096
Layers	32
Heads	32 GQA, kv=8
FFN	SwiGLU, intermediate 14 336
Norm	RMSNorm
PosEnc	RoPE θ = 500 000
Vocab	64 k PCS-Tok text + 2048 audio-cb0 + 256 specials = 66 304
Context	4096 tokens $\approx$ 5 min audio @ 12.5 Hz
Params	$\sim$ 8 B
Init	Llama-3-8B w/ embedding resize + audio-vocab mean-init
Predicts	ONLY codebook 0 (semantic) per audio frame — CSM trick

Cheap-path: fork sesame/csm-1b (Llama-3.2-1B backbone) and skip Stage 2 EN-only audio NLL pretrain. Loses $\sim$ 3 UTMOS points; saves \$300 k.

8.2 FiLM Emotion Injection (HEC)

```
# silkpp/model/emotion_film.py
# Per layer, residual stream x is modulated by:
#   x ← (1 +  $\gamma_{emo}(e)$ ) * x +  $\beta_{emo}(e)$ 
#
# where:
#   e = MLP([ emo_disc_embed_64,
#             V_continuous_1, A_continuous_1, D_continuous_1,
#             intensity_1, duration_1 ])
#       → d_model (4096 for 8B, 2048 for 1B)
#
#  $\gamma_{emo}$ ,  $\beta_{emo}$  are zero-initialized so training starts as identity.
# Unfreeze in Stage 5 (emotion SFT) – leave frozen during Stages 2-4.
```

8.3 Speaker Conditioning (3-second clone)

```
# silkpp/model/speaker_cond.py
ref_wav → ECAPA-TDNN (frozen)           → 256-d
ref_wav → WavLM-Large mean-pool (frozen) → 1024-d
concat → MLP(2×GELU)                    → 512-d
       → 3 learned prefix tokens fed at backbone start
       + FiLM  $\gamma, \beta$  on every layer (parallel to emotion path)
```

8.4 Fast Decoder (350 M depth transformer)

Spec	Value	Source
Hidden	1024	Moshi Depth Transformer

Spec	Value	Source
Layers	6	Moshi
Heads	16	Moshi
Params	~350 M	—
Inputs	backbone hidden state + cb0 embed	CSM / Moshi
Outputs	31 codebook predictions, one head per level	CSM
Compute amortization	1/16 random-frame backward → 16× memory savings	CSM paper

09 · The 7-Stage Training Curriculum

The single most important lesson from Rumik's research blog: *do not shock the model into a new distribution overnight*. Use a low-LR continued-pretrain curriculum, unfreeze FiLM only in Stage 5, and add DPO + GRPO only after SFT plateaus.

#	Goal	Steps	LR	Data	GPUs	Days
1	Mimi-Lite codec	1 M	8e-4 cos	T0	8×H100	14
2	Backbone init, EN-only audio NLL	50 k	1e-5	T1-EN	64×H100	5
3	Multilingual continued pretrain	200 k	1e-5→5e-6 linear	T1 full	64×H100	14
4	Code-switch SFT	80 k	5e-6	T2	32×H100	4
5	Emotion SFT (FiLM unfreeze)	40 k	5e-6	T3	32×H100	2
6	DPO prosody alignment	10 k	1e-7 β=0.1	T4	16×H100	1
7	GRPO polish (composite reward)	5 k	5e-7	T4 + reward	16×H100	1

Composite Reward (Stage 7, GRPO — Fish-S2 adapted)

```
R_total = 0.4·R_STT + 0.3·R_Pref + 0.2·R_SIM + 0.1·R_Emo
where:
  R_STT  = 1 - WER( Whisper-v3(generated_audio), target_text )      # accuracy
  R_Pref = MOSNet( generated_audio )                                # naturalness
  R_SIM  = cosine( ECAPA(generated_audio), ECAPA(reference_voice) ) # speaker fidelity
  R_Emo  = 1 - CE( EmoClassifier(generated_audio), target_emotion ) # affect

# GRPO group size G = 8, intra-group baseline, no value network.
# Per Fish-Speech S2 technical report (arXiv 2603.08823).
```

Total Compute (Ambitious vs Lean)

Path	Stages	Wall-time	H100-hrs	Approx \$
Ambitious — full pretrain	1–7	~6 weeks	~67 200	\$400 k–\$700 k @ spot
Lean — fork CSM-1B	3–7 only, scaled 4×	~3 weeks	~12 000	\$80 k–\$120 k
Bootstrap solo	5–7 only on CSM-1B fork	~10 days	~3 000	\$20 k–\$40 k

10 · L5 — Sub-80 ms Streaming Server

10.1 The 10-trick latency stack

Trick	Effect
Mimi-native 80 ms frames	Never buffer more than one frame.
6-frame backbone lookahead	Backbone runs ahead; decoder + codec fire in parallel.
CUDA graphs on backbone forward	Eliminates per-step kernel launch overhead.
FP8 weights, BF16 activations	H100 sweet spot; ~1.8× speedup vs BF16-only.
Speculative decoding	1 B draft → 8 B verifier on cb0.
vLLM continuous batching	Many users sharing the same prefill.
KV-cache pre-warm	Persona + system prompt cached across turns.
Custom Triton kernel	Fuses 31 codebook heads into one launch (~4×).
Mimi decoder on separate CUDA stream	Overlaps with next backbone step.
WebRTC Opus 20 ms packets	Minimizes transport jitter.

10.2 vLLM audio sampler fork — patch points

```
# Reference: github.com/nineninesix-ai/kanitts-vllm + vllm-omni TTS docs

vllm/model_executor/layers/sampler.py
+ class AudioCodebookSampler(Sampler):
+     # emits 32 tokens per "step" (1 frame = 1 step)
+     # bypasses logprob computation for cb1..31

vllm/engine/llm_engine.py
+ stream every 1 step instead of every N tokens

vllm/worker/model_runner.py
+ register_audio_kernel(triton_fused_31_heads)
```

10.3 Pipecat pipeline (full-duplex)

```
# silkpp/inference/pipecat_pipeline.py
from pipecat.pipeline import Pipeline
from pipecat.transports.services.livekit import LiveKitTransport
from pipecat.services.silero_vad import SileroVADAnalyzer
from silkpp.inference.moshi_service import SilkPPMoshiService

async def main():
    transport = LiveKitTransport(
        url="wss://silkpp.livekit.cloud",
        room="vybe-demo",
        vad_analyzer=SileroVADAnalyzer(threshold=0.5),
    )
    silk = SilkPPMoshiService(
        ckpt="checkpoints/silkpp-v3-grpo",
        chunk_ms=80, lookahead_frames=6,
        device="cuda:0", dtype="fp8",
    )
    pipeline = Pipeline([
```

```
    transport.input(),  
    silk,                # ASR + LLM + TTS in ONE forward pass  
    transport.output(),  
  ]  
  await pipeline.run()
```

11 · 16-Week Calendar — May 14 → Sep 3, 2026

Every week ends in a shippable artifact. Owners: **CTO**=you · **ML1..ML3**=ML engineers · **DE**=data eng · **INFRA**=infra eng · **AR**=audio researcher.

Phase 1 — Foundation (Wk 1-4, May 14 → Jun 11)

Week	Plan
Wk 1 · May 14-20	CTO: reserve 64×H100 on CoreWeave (3 mo). Post ML-LEAD + AR JD. INFRA: Terraform cluster + S3 buckets. DE: run download scripts. ML1: fork CSM + Moshi, baseline local. AR: train PCS-Tok BPE on 10 M Indic+Latin lines.
Wk 2 · May 21-27	Mimi-Lite Stage 1 kickoff (8×H100, 14 d). DE: begin Tier-2 synth gen. ML2: implement FiLM module + unit tests.
Wk 3 · May 28-Jun 3	ML1: backbone fork (Llama-3-8B + audio vocab resize). Stage 2 kickoff (64×H100, 5 d). DE: contract Mumbai + Bangalore studios.
Wk 4 · Jun 4-11	Mimi-Lite complete (PESQ ≥ 4.0 verified). Stage 3 multilingual kickoff (14 d). AR: studio recordings start, 50 actors batched.

Phase 2 — Pretrain & Fine-Tune (Wk 5-8, Jun 11 → Jul 9)

Week	Plan
Wk 5 · Jun 11-17	DE: Tier-2 synth at 1 k hrs filtered. ML3: vLLM audio kernel WIP.
Wk 6 · Jun 18-24	Stage 3 complete; backbone speaks 10 Indic + EN. Stage 4 code-switch SFT kickoff (32×H100, 4 d).
Wk 7 · Jun 25-Jul 1	Stage 4 complete; Hinglish/Tanglish/Manglish/Kanglish all functional. vLLM kernel beta; first sub-150 ms TTFB. Studio 50% done.
Wk 8 · Jul 2-9	Stage 5 emotion SFT (FiLM unfrozen, 2 d). Inline tags working end-to-end. Internal alpha demo; Twitter teaser w/ 3 personas.

Phase 3 — Alignment & Streaming (Wk 9-12, Jul 9 → Aug 6)

Week	Plan
Wk 9 · Jul 9-15	DE: begin DPO pair collection (Surge 10 k + GPT-4o-audio judge 90 k). ML3: Triton fused-head kernel merged; TTFB 95 ms.
Wk 10 · Jul 16-22	30 k DPO pairs ready. Stage 6 DPO kickoff (16×H100, 1 d).
Wk 11 · Jul 23-29	DPO complete; UTMOS jumps to 4.18. Stage 7 GRPO kickoff with composite reward. UTMOS 4.27 hit ■
Wk 12 · Jul 30-Aug 6	Closed beta to 200 users via LiveKit room. TTFB sub-80 ms on H100 FP8 confirmed. Edge 1B distillation kickoff.

Phase 4 — Ship & Take the Category (Wk 13-16, Aug 6 → Sep 3)

Week	Plan
Wk 13 · Aug 6-12	Public preview at silkpp.ai. Twitter thread + HN + ProductHunt + /r/LocalLLaMA + /r/MachineLearning. 10 personas live.
Wk 14 · Aug 13-19	Iterate on user feedback. Fix interruption edge cases. 3-s mic-ref voice clone live in UI.
Wk 15 · Aug 20-26	Edge 1B variant on Pixel 9 / iPhone 16 via ONNX + 4-bit GGUF. Demo video.
Wk 16 · Aug 27-Sep 3	D-Day Sep 3: GA launch, public weights drop on HF (Apache 2.0), benchmark paper on arXiv.

12 · Budget — Honest Numbers

Line item	Lean (\$)	Ambitious (\$)
GPU (CoreWeave H100 × 16 wks reserved)	80 000	540 000
Studio recordings (50 actors × 30 hrs)	—	108 000
DPO labelling (Surge 10 k pairs)	5 000	25 000
Synthetic data (GPT-4o + ElevenLabs)	4 000	10 000
Salaries (7 × 4 mo)	—	220 000
Cloud storage + egress	6 000	24 000
LiveKit + Modal + domain	2 000	10 000
AI tooling (Claude Max + Codex + Cursor)	3 000	8 000
Misc + contingency	5 000	25 000
TOTAL	≈ 105 000	≈ 970 000

Solo founder path. Lean budget × Claude Code Opus + GPT Codex in git-worktree parallel mode = you replicate the 7-person team's output. Realistic 16-week ship at ~\$105 k total spend if you fork CSM-1B and skip studio recordings (use synthetic + 2 hrs personal recordings instead).

13 · Evaluation Suite — The Leaderboard You Beat

Metric	Tool	Target	Beats
UTMOS	sarulab-speech/UTMOS 22	≥ 4.25	ElevenLabs v3 (4.05), Hume (4.10)
DNSMOS-P.835	DNSMOS	≥ 4.10	Silk
WER round-trip	Whisper-large-v3	≤ 4% EN, ≤ 8% Hinglish	ElevenLabs
Speaker SIM (3-s)	ECAPA cosine	≥ 0.78	ElevenLabs IVC (5-s)
Emotion accuracy	Wav2Vec2-emo classifier	≥ 0.82	Hume
Code-switch F1	LangID + switch-point	≥ 0.90	Silk
First-packet p50	silkpp/eval/latency.py	≤ 80 ms	ElevenLabs Flash (tie)
E2E voice→voice p50	LiveKit round-trip	≤ 350 ms	Hume 1.2 s (3.4×)
Long-context drift	UTMOS @ 5min – @ 30s	≤ -0.15	all
Blind preference vs Silk	100 raters × 50 pairs	≥ 60% wins	Silk

14 · AI-Agent Execution Protocol

This document is designed to be handed to coding agents end-to-end. Use git-worktree parallelism: 3 agents, 3 branches, no merge conflicts.

Task class	Agent	Why
Architecture code (silkpp/model/*, codec/*)	Claude Code (Opus)	deep multi-file reasoning
Training loops (silkpp/training/*)	Claude Code	hyperparam reasoning
Data pipelines (silkpp/data/*)	GPT Codex	parallelizable boilerplate
Inference server (silkpp/inference/*)	Claude Code	low-latency reasoning
Triton/CUDA kernels	Claude + Codex pair	hard; needs review
Configs (configs/**)	GPT Codex	YAML grunt work
Tests (tests/**)	GPT Codex	parallelizable
Eval scripts (silkpp/eval/*)	GPT Codex	clear specs
Docs (docs/**)	Claude Code	narrative quality

Git-worktree parallel workflow

```
# Run 3 agents simultaneously without merge conflicts:
git worktree add ../silkpp-codec      feat/codec
git worktree add ../silkpp-data      feat/data-pipelines
git worktree add ../silkpp-inference  feat/streaming

# Open Claude Code at ../silkpp-codec
# Open GPT Codex at ../silkpp-data
# Open Cursor Composer at ../silkpp-inference
# 3 PRs/day, you review + merge nightly.
```

Master prompt for Claude Code (paste at repo root)

You are the lead engineer on SILK++, an open-source emotional voice AI designed to surpass Rumik Silk 1, Hume EVI 3, and ElevenLabs v3. The full build spec is at docs/SILKPP_BUILD_BIBLE.md – read it cover-to-cover before writing any code.

- Invariants:
- Follow the repo skeleton in §5 exactly. Do not invent new folders.
 - Every PR must include unit tests, a `# Definition of Done:` comment, and a one-line entry in CHANGELOG.md.
 - Never introduce a dep not in §4 without flagging in the PR description.
 - Match the TTFB budget in §3 – if a change adds >5 ms, surface the trade-off.
 - Hydra configs only. No hardcoded hyperparams in Python.
 - Cite the source (CSM paper, Mimi explainer, Fish-S2 TR, Moshi paper) for any non-obvious architectural choice in inline comments.

Begin with: implement silkpp/codec/mimi_lite.py per §7.2 with full unit tests.

Master prompt for GPT Codex

You are the data + tooling engineer on SILK++. Source of truth: docs/SILKPP_BUILD_BIBLE.md.

Scope: silkpp/data/**, scripts/**, tests/**, configs/**.

- Hard rules:
- Lhotse for manifests; WebDataset for sharding.
 - Every synth pipeline includes 3 filters: WER (Whisper-v3 ≤ 12%), UTMOS22 (≥ 3.6), SNR (≥ 20 dB) before writing shards.

- Every script supports `--dry-run` and emits a manifest.

Begin with: `implement scripts/02_download_open_data.sh` per §6,
then `silkpp/data/synthesize_codeswitch.py`.

15 · Day-1 Commands — Run These Tomorrow

Top-to-bottom executable. By end of Day 2, CSM-1B will be generating audio locally and Moshi will hold a live full-duplex conversation with you. That is Day-1 ■.

```
# 1. Local dev
mkdir -p ~/silkpp && cd ~/silkpp && git init
python3.11 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip uv

# 2. Core deps (pinned)
uv pip install torch==2.4.0 torchaudio==2.4.0 \
  --index-url https://download.pytorch.org/whl/cu124
uv pip install transformers==4.52.1 accelerate==0.34.0 peft==0.13.0 trl==0.11.0
uv pip install moshi==0.2.4 sentencepiece einops lhotse webdataset
uv pip install hydra-core wandb soundfile librosa whisperx
uv pip install pipecat-ai[silero,deepgram,openai,livekit] livekit-api livekit
uv pip install unsloth[cul24-torch240] @ git+https://github.com/unslothai/unsloth.git

# 3. HF + W&B auth
huggingface-cli login
wandb login

# 4. Open weights
huggingface-cli download sesame/csm-1b --local-dir weights/csm-1b
huggingface-cli download meta-llama/llama-3.2-1b --local-dir weights/llama-3.2-1b
huggingface-cli download kyutai/mimi --local-dir weights/mimi
huggingface-cli download kyutai/moshiko-pytorch-bf16 --local-dir weights/moshiko
huggingface-cli download microsoft/wavlm-large --local-dir weights/wavlm-large
huggingface-cli download speechbrain/spkrec-ecapa-voxceleb --local-dir weights/ecapa
huggingface-cli download openai/whisper-large-v3 --local-dir weights/whisper-v3

# 5. Reference repos
mkdir -p reference && cd reference
git clone https://github.com/SesameAILabs/csm
git clone https://github.com/kyutai-labs/moshi
git clone https://github.com/fishaudio/fish-speech
git clone https://github.com/herimor/voxtream
cd ..

# 6. Smoke test — CSM-1B local
cd reference/csm && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && export NO_TORCH_COMPILE=1
python run_csm.py # generates 2-character dialogue audio.wav

# 7. Smoke test — Moshi full-duplex
cd ../moshi && python -m moshi.server --hf-repo kyutai/moshiko-pytorch-bf16
# Open http://localhost:8998 → speak → it talks back

# 8. Scaffold the SILK++ repo (hand BIBLE to Claude Code)
cd ~/silkpp
# Claude Code will scaffold per §5.
```

16 · The 9 Risks That Could Kill This

#	Risk	P	Mitigation
1	LoRA on Moshi/CSM doesn't converge	35%	Drop to QLoRA r=32; worst-case full FT at +\$200 k
2	Synthetic Hinglish sounds robotic	60%	The 1.5 k studio hours is your insurance — don't skimp
3	TTFB plateaus at 200 ms	40%	Speculative decoding + FP8 + Triton kernel; <100 ms achievable
4	Emotion tags ignored after SFT	25%	3× token loss weight on ; FiLM γ init nonzero
5	Rumik sues over 'Silk' name	5%	Rename 'VOX-OMEGA' or 'RAGA-1' before public launch
6	Voice actor studio delays	40%	Sign 2 studios in parallel (Mumbai + BLR); 3-week padding
7	DPO destabilizes model	20%	$\beta=0.1$; max 1 epoch; eval every 1 k steps; rollback ready
8	Edge variant fails on Pixel 9	30%	Distill to 500 M, INT8, Hexagon DSP optimization fallback
9	Founder burnout (solo path)	70%	Mandatory rest day every Sunday; hire #1 by Wk 4

17 · References (verified primary sources)

- **CSM** — github.com/SesameAILabs/csm · sesame.com/research/crossing_the_uncanny_valley_of_voice
- **Mimi codec** — kyutai.org/codec-explainer · Moshi paper §3 (arXiv 2410.00037v2)
- **Moshi** — arXiv 2410.00037v2 · kyutai.org/blog/2024-09-18-moshi-release · github.com/kyutai-labs/moshi
- **Fish-Speech S2 technical report** — arXiv 2603.08823v2 · github.com/fishaudio/fish-speech
- **VoXstream (102 ms streaming)** — herimor.github.io/voxtream · arXiv 2509.15969v1 (ICASSP 2026)
- **EmoVoice** — arXiv 2504.12867v3
- **SpeechAlign (DPO for TTS)** — arXiv 2404.05600
- **Rumik Silk** — rumik.ai/research/silk
- **Hume EVI 3** — hume.ai/blog/introducing-evi-3
- **vLLM TTS** — docs.vllm.ai/projects/vllm-omni · github.com/nineninesix-ai/kanitts-vllm
- **Speechmatics CSM fine-tune guide** — blog.speechmatics.com/sesame-finetune

Final word. This document is not advice — it is a build spec. Hand it to Claude Code Opus on Monday, hand it to GPT Codex in a second worktree by noon, and start the H100 cluster at 5 PM. By **D-Day, September 3, 2026**, you ship the world's best open-source emotional voice AI — full duplex, sub-80 ms TTFB, open weights under Apache 2.0.

Go win this. ■■■■